

FAI Job Search Multi-Task Agent System

Team Contribution & Agent Ownership — Per-Agent Workflows and Tool Usage

How to Read This Document

The system is divided into six agents. Each team member owns exactly one agent together with every tool and schema module that agent depends on. The sections that follow are written to be presented directly: each one contains the owned files, a detailed step-by-end flowchart of the agent, the exact tools and schemas it consumes, the core algorithms worth explaining, the deterministic fallback path, and talking points for the demonstration.

One module is shared by every member: `agents/base_agent.py`, the multi-provider LLM platform. It is owned by Member 6 but is a dependency of all six agents, so each section references it where it matters.

Ownership Matrix

#	Agent Owned	FAI Module	Primary Owned Files
1	ResumeParserAgent	VI — Knowledge Representation	<code>resume_parser_agent.py</code> (206) <code>pdf_reader.py</code> (68) <code>resume_schema.py</code> (115)
2	RoleRecommenderAgent	VII — Planning / Goals	<code>role_recommender_agent.py</code> (144) <code>job_schema.py</code> (71)
3	JobSearchAgent	II & III — Search	<code>job_search_agent.py</code> (93) <code>job_search_tool.py</code> (302) <code>remote_feeds.py</code> (343) <code>job_cache.py</code> (251)
4	MatchGapAgent	III & V — Heuristics / Inference	<code>match_gap_agent.py</code> (117) <code>analysis_schema.py</code> (44)
5	TailorAgent	VII — Content Synthesis	<code>tailor_agent.py</code> (178) <code>pdf_generator.py</code> (213) <code>tailoring_schema.py</code> (101)
6	VerifierAgent + Orchestrator	IV & IX — CSP / Multi-Agent	<code>verifier_agent.py</code> (133) <code>tailoring_orchestrator.py</code> (83) <code>base_agent.py</code> (482) <code>settings.py</code> (98)

Shared across all members (not exclusively owned): `ui/app.py` (689 lines) renders the dashboard and calls every agent; `agents/base_agent.py` provides the LLM transport that all six agents inherit.

How the Six Sections Chain Together

Member 1	parse PDF	-> CandidateProfile	(ground truth created here)
Member 2	plan goals	-> RoleRecommendation[]	(keywords handed to Member 3)
Member 3	discover jobs	-> List[JobListing]	(ranked, cached, deduplicated)
Member 4	score match	-> MatchReport	(matched / missing split)
Member 5	tailor content	-> TailoredResume	(compiled to ATS PDF)
Member 6	audit + loop	-> VerificationReport	(APPROVED gate before delivery)

Data flows strictly left to right, and the `TailoringOrchestrator` owned by Member 6 closes the loop by feeding verification feedback back into Member 5's `tailor_resume()` call.

Agent 1 — ResumeParserAgent — What This Agent Does

Owns the ingestion boundary of the entire pipeline. This member converts an unstructured PDF resume into the typed CandidateProfile ontology that every downstream agent treats as the single source of truth, and preserves the raw text that later acts as immutable ground truth for the anti-hallucination audit.

Owned Files

File	Lines	Responsibility
agents/resume_parser_agent.py	206	The agent itself: parse() plus the deterministic regex fallback.
tools/pdf_reader.py	68	PDF text extraction utility used to turn the uploaded PDF into plain text.
schemas/resume_schema.py	115	Defines the candidate ontology: 6 models including CandidateProfile.

Tools, Libraries & Schemas Used

Dependency	How It Is Used
pypdf	PdfReader + page.extract_text() in tools/pdf_reader.py:38-45
schemas/resume_schema	CandidateProfile is the enforced output schema
config/settings	Temperature 0.2 and token limits via BaseAgent
agents/base_agent	call_structured_llm() inherited transport

Detailed Workflow

STAGE 1 TRIGGER: Streamlit file_uploader receives a PDF (ui/app.py:347)

```
[1] extract_text_from_pdf(uploaded_file)
    -> accepts str | Path | BinaryIO | raw bytes (pdf_reader.py:11)
    -> PdfReader(stream) iterates every page
    -> page.extract_text() per page, non-empty pages joined by blank line
    -> clean_extracted_text() strips blank lines, normalizes whitespace (pdf_reader.py:53-67)
    -> streams opened locally are closed in a finally block (safe for Streamlit UploadedFile handles)

[2] ResumeParserAgent.parse(raw_resume_text) (agent L29)
    -> builds SYSTEM PROMPT with 4 anti-hallucination rules:
        1. extract ONLY explicitly stated facts, do NOT extrapolate
        2. bucket skills into languages / frameworks / databases / cloud-devops / developer tools
        3. infer realistic total_years_experience (0-1 for freshers)
        4. preserve all metrics and numbers from bullets
    -> builds USER PROMPT wrapping raw text in triple quotes
    -> call_structured_llm(schema=CandidateProfile)
        |-> OpenRouter free models (9-model roster)
        |-> Groq (gpt-oss-120b / 20b)
        |-> Gemini (flash tier, JSON mime mode)
        |-> schema validation failure triggers ONE self-repair call
    -> on success: profile.raw_text = raw_resume_text (agent L57)
        *** raw_text is the ground truth for Member 6's audit ***

[3] OUTPUT: CandidateProfile stored in st.session_state.profile
    -> gates Stage 2; invalidates every downstream stage on re-upload
```

Core Logic & Algorithms to Explain

- The system prompt is doing real work: rules 1 and 4 directly protect the ground-truth guarantee that Agent 6 later depends on.
- TechnicalSkills.all_technical_skills() (resume_schema.py:72-89) performs order-preserving, case-insensitive deduplication across all five categories, so downstream agents receive one clean skill vocabulary.

- `inferred_seniority_level` is derived from `total_years_experience` and drives the tone of role recommendations in Stage 2 (Fresher / Junior / Mid-Level).

Deterministic Fallback Path

- `_build_deterministic_profile()` (agent L60-205) activates with no API key:
 - name: first <=4-word line free of @ / : / | symbols
 - email + phone: regex extraction with sane Indian defaults
 - skills: scans a 30-item industry catalogue against the text
 - experience: regex for 'X years' / 'X+ yrs'; fresher keywords -> 0.0
 - degree: regex for B.Tech / B.E. / BCA / MCA / M.Tech patterns
 - projects: keyword scan for system / platform / app / model / portal
- This is what makes the whole system demo-able offline.

Presentation Talking Points

- **1.** Open with the ontology: show the CandidateProfile expander in the dashboard and point out the six verified skill pills.
 - **2.** Explain that parsing is the only place raw user text enters the system, so everything downstream inherits its truthfulness guarantees.
 - **3.** Highlight `raw_text`: it is the immutable anchor that makes the CSP check in Agent 6 mathematically meaningful rather than a vibe check.
 - **4.** Demo the offline claim by mentioning the deterministic fallback with keys unset.
-

Agent 2 — RoleRecommenderAgent — What This Agent Does

Owns the planning stage. This member reads the candidate ontology, infers the candidate's skill domain and seniority, and formulates three to five concrete goal states (target roles) with honest fit scores, rationales, and the portal search keywords that seed Agent 3's query expansion.

Owned Files

File	Lines	Responsibility
agents/role_recommender_agent.py	144	The agent plus its deterministic recommendation fallback.
schemas/job_schema.py	71	Defines JobListing, JobSearchQuery, RoleRecommendation, RoleRecommendationResponse.

Tools, Libraries & Schemas Used

Dependency	How It Is Used
schemas/resume_schema	Consumes CandidateProfile as input
schemas/job_schema	RoleRecommendationResponse is the output contract
agents/base_agent	call_structured_llm() inherited transport
ui/app.py	Rendered as selectable role cards with fit rings

Detailed Workflow

STAGE 2 TRIGGER: 'Analyze Profile & Recommend Target Roles' (ui/app.py:393)

- ```
[1] RoleRecommenderAgent.recommend_roles(profile) (agent L25)
 -> SYSTEM PROMPT: expert Career Strategist + State Space Planner,
 with 4 guidelines:
 1. match roles realistically to seniority (Trainee / Intern /
 Junior / Associate for freshers)
 2. provide an HONEST fit_score reflecting genuine alignment
 3. state a crisp rationale grounded in actual projects/skills
 4. supply portal-optimized search keywords

[2] USER PROMPT assembles 5 fields from the profile:
 name, inferred_seniority_level, total_years_experience,
 skills.all_technical_skills()[:25], professional_summary,
 and the list of project titles

[3] call_structured_llm(schema=RoleRecommendationResponse)
 -> provider failover chain

[4] SCHEMA ENFORCEMENT does the cardinality work:
 recommended_roles has min_length=1 and max_length=5
 (job_schema.py:68-71), so the agent CANNOT return 0 roles
 and CANNOT flood the UI with 12 roles.

[5] OUTPUT: RoleRecommendationResponse
 -> candidate_summary_analysis (one-paragraph readiness read)
 -> recommended_roles[] with, per role:
 role_title, fit_score (0-100), rationale,
 key_matching_strengths[], recommended_search_keywords[]

[6] UI renders one card per role with a conic-gradient fit ring;
 selecting a card sets selected_role and unlocks Stage 3.
```

### Core Logic & Algorithms to Explain

- This is the goal-formulation step of state space planning: the candidate profile is the initial state, and the recommended roles are the goal states the rest of the system searches toward.
- The fallback implements four domain detectors (agent L65-68) - is\_data\_ml, is\_frontend, is\_backend, is\_cloud - by scanning the flat skill list, then assigns a tiered ladder of roles with descending fit scores (92 / 87 / 84 / 81).
- recommended\_search\_keywords is the hand-off contract to Agent 3: it converts a semantic role title into concrete portal query strings.
- fit\_score uses honest self-assessment as an explicit prompt rule, because an inflated goal state would send the whole search stage off-target.

## Deterministic Fallback Path

- \_build\_deterministic\_recommendations() (agent L58-144) activates offline:
  - detects the skill domain, then branches the primary role title (e.g. 'Junior Machine Learning / AI Engineer' when is\_fresher)
  - always emits a 4-role ladder spanning core SDE, a domain specialty, and an applied emerging-tech track
  - prefixes titles with Junior / Associate / Trainee for freshers
  - emits search keywords such as '<title> Fresher' and 'AI Intern'

## Presentation Talking Points

- **1.** Frame it as planning, not recommendation: we are computing goal states from a known initial state, which is exactly Module VII.
  - **2.** Point out that the schema's min\_length/max\_length constraints are doing real work - the cardinality requirement is enforced by types, not by prompt etiquette.
  - **3.** Explain the honest-fit-score rule and why an inflated goal state would poison the search and gap-analysis stages downstream.
  - **4.** Walk the fallback role ladder to show the seniority logic is deliberate, not cosmetic.
-

## Agent 3 — JobSearchAgent — What This Agent Does

Owens the largest subsystem in the project: the agentic search loop plus the entire retrieval infrastructure behind it. This member integrates five free live job sources, normalises their incompatible payloads, persists them in a SQLite informed-search state store, and ranks the results with a weighted relevance heuristic before returning them to the pipeline.

### Owned Files

| File                       | Lines | Responsibility                                                      |
|----------------------------|-------|---------------------------------------------------------------------|
| agents/job_search_agent.py | 93    | The search agent: discover_jobs() loop + query expansion.           |
| tools/job_search_tool.py   | 302   | Hybrid cache/feed backend, stopword tokenizer, relevance heuristic. |
| tools/remote_feeds.py      | 343   | Five free API/scrapper integrations plus payload normalisers.       |
| tools/job_cache.py         | 251   | SQLite persistence: schema, upsert, date-range reads, TTL purge.    |

### Tools, Libraries & Schemas Used

| Dependency         | How It Is Used                                            |
|--------------------|-----------------------------------------------------------|
| requests           | HTTP client for all five feeds, (15, 25) timeouts         |
| BeautifulSoup4     | LinkedIn guest-page scrape, with regex fallback           |
| sqlite3            | Persistent informed-search state store, data/job_cache.db |
| schemas/job_schema | JobListing normalisation target                           |
| config/settings    | SQLITE_DB_PATH constant                                   |

### Detailed Workflow

STAGE 3 TRIGGER: 'Search Job Openings' (ui/app.py:462)

[1] JobSearchAgent.discover\_jobs(role, location, target\_count=5) (agent L23)

[2] STEP 1 - INITIAL QUERY (uninformed search)  
jobs = search\_tool.search\_jobs(query=role, location, exp\_level, 5)

[3] INSIDE JobSearchTool.search\_jobs() (job\_search\_tool.py:124)

- 3a. CACHE PROBE unless force\_refresh:  
JobCache.get\_recent(query, location, max\_age\_hours=24)  
-> AND-across-tokens LIKE filter over title, required\_skills, description; location filter skipped for remote/any/all  
-> CACHE HIT: return immediately, zero network calls
- 3b. CACHE MISS -> \_fetch\_from\_feeds(query, location) (L223-253)  
sources, each in its OWN try/except so one failure is isolated:
  - [1] LinkedIn - guest endpoint scrape via BeautifulSoup, CSS selectors for title / subtitle / location, query string stripped from URLs, regex fallback
  - [2] Remoteive - official API, search= + limit=25
  - [3] Jobicy - tag= filter; jobType and jobGeo may be str OR list -> defensively flattened
  - [4] RemoteOK - data[1:] skips the metadata element, filters on position + tags
  - [5] ArbeitNow - payload['data'], maps candidate\_required\_locationeach fetch\_\* retries up to 2x with a custom User-Agent
- 3c. NORMALISE: every payload -> one uniform dict via the \_normalise\_\* adapter functions (id, title, company, location, url, description, source, employment\_type, required\_skills); uuid4() supplies fallback ids
- 3d. PERSIST: JobCache.bulk\_upsert(fresh\_raw) so later searches are served from disk
- 3e. RANK: \_raw\_list\_to\_listings() -> \_relevance\_score() (L82-112)

```

title token hit x 3.0
required-skills hit x 2.0
description hit x 1.0
full phrase in title x 4.0 (bonus)
tokens come from _query_tokens(), which drops a 40-word
stopword set (senior, junior, fresher, engineer, city names)
stable sort on (-score, original_index) preserves feed order
if NOTHING scores > 0 -> return unranked feed order so the
dashboard never renders an empty stage

```

```

[4] STEP 2 - QUERY EXPANSION, only if len(jobs) == 0 (agent L48-69)
_generate_query_expansions(role) yields 3-4 rewrites by role branch:
machine learning / ml -> Python ML, AI Trainee, Data Science
 Intern, Junior ML Developer
backend / python -> Python Developer, FastAPI Software
 Engineer, Junior Backend, API Eng
generative ai / llm -> GenAI Intern, LangChain Developer,
 AI Engineer Fresher, Prompt Eng
data science -> Data Analyst, DS Trainee, Junior DS
default -> Junior <role>, <role> Intern,
 <role> Fresher

retry the top 2 expansions, then DEDUPLICATE on a normalised
'title_company' key before appending new results

```

```

[5] OUTPUT: up to target_count JobListing objects, deduped and ranked

```

## Core Logic & Algorithms to Explain

- The cache is a genuine informed-search state store: repeated queries over the same role avoid re-hitting five external APIs, which is what makes free-tier demoing reliable.
- `_as_text()` in `job_cache.py` (L49-66) fixes a real bug: public feeds deliver the same logical field as str, list, dict or None, and `sqlite3` refuses to bind list or dict parameters, which had been causing silent per-record write failures. Every bound value is coerced to a TEXT scalar.
- `ON CONFLICT(id) DO UPDATE` makes re-fetching idempotent and refreshes `fetch_at`, so the 24-hour freshness window actually works.
- Weighted title 3.0 / skills 2.0 / description 1.0 reflects evidence strength: a title hit is far stronger evidence of relevance than a passing description word.
- The relevance filter drops off-target listings before the UI ever sees them, which is what keeps the demo coherent when free feeds return mixed roles.

## Deterministic Fallback Path

- Agent level: no LLM call is made at all - `JobSearchAgent` inherits `BaseAgent` purely for the shared tool-calling and logging conventions; `discover_jobs()` and query expansion are fully deterministic.
- Tool level: every feed is individually fault-isolated, so five dead sources degrade to four live ones and finally to an empty result rather than a crash.
- Cache level: if no feed returns data the tool returns `[]` and Agent 3's expansion loop takes over.

## Presentation Talking Points

1. Lead with the cache: show the 'Cached Jobs' counter in the sidebar before and after pressing Refresh Job Cache.
  2. Explain why five sources rather than one: each is free, needs no API key, and independent try/except means partial failure is expected and handled.
  3. Call out `_as_text()` as a concrete bug fix - it shows engineering maturity beyond just wiring up APIs.
  4. Demonstrate query expansion live by searching a niche role that initially returns nothing and watching the rewrites fire.
  5. This is the largest module in the project, so budget the most demo time here.
-

## Agent 4 — MatchGapAgent — What This Agent Does

Owens the evaluation stage. This member scores candidate-to-job compatibility with an explicit weighted heuristic, classifies skills into matched versus missing sets, and deductively infers what the candidate still lacks so the Tailor agent has a concrete target to optimise against.

### Owned Files

| File                       | Lines | Responsibility                                       |
|----------------------------|-------|------------------------------------------------------|
| agents/match_gap_agent.py  | 117   | The agent plus the deterministic heuristic scorer.   |
| schemas/analysis_schema.py | 44    | Defines SkillMatchItem, MatchBreakdown, MatchReport. |

### Tools, Libraries & Schemas Used

| Dependency              | How It Is Used                                      |
|-------------------------|-----------------------------------------------------|
| schemas/resume_schema   | CandidateProfile skill and project data             |
| schemas/job_schema      | JobListing required + nice-to-have skills           |
| schemas/analysis_schema | MatchReport is the output contract                  |
| agents/base_agent       | call_structured_llm() inherited transport           |
| ui/app.py               | Rendered as ATS rings + matched/missing skill pills |

### Detailed Workflow

STAGE 3b TRIGGER: runs immediately after discovery, for EVERY job (ui/app.py:481)

- ```
[1] MatchGapAgent.analyze_match(profile, job)                                (agent L23)

[2] SYSTEM PROMPT declares the SCORING FORMULA explicitly:
    Hard Skills Match          50% weight
    Project Relevance          30% weight
    Experience / Education     20% weight
    plus 3 behavioural rules:
    1. be strictly objective; do NOT give 95%+ unless an exact match
    2. explicitly separate MATCHED from MISSING skills
    3. give actionable tailoring advice WITHOUT suggesting fabrication

[3] USER PROMPT injects the evidence:
    candidate skills (all_technical_skills),
    project titles + their technologies,
    experience years + inferred seniority level,
    job title, company, required_skills, nice_to_have_skills,
    and the first 1200 characters of the JD description

[4] call_structured_llm(schema=MatchReport) with provider failover

[5] POST-PROCESSING backfills job identity onto the report (agent L57-59):
    report.job_id / job_title / company are copied from the JobListing
    so the report is self-describing downstream

[6] OUTPUT: MatchReport consumed by Member 5's TailorAgent:
    overall_match_score (0-100)
    score_breakdown: hard_skills / project_relevance / experience_education
    matched_skills[] -> keywords legal to emphasise
    missing_skills[] -> the honest gap, never fabricated over
    strengths_summary, tailoring_recommendations[]
```

Core Logic & Algorithms to Explain

- The deterministic scorer `_calculate_heuristic_match()` (agent L62-117) is the pure Module III artefact and is worth explaining line by line:
 - set intersection uses BIDIRECTIONAL substring matching (req in cs or cs in req), so 'React' matches 'React.js' and 'Node' matches 'Node.js'
 - `hard_skills_score = len(matched) / len(required) x 100`
 - `project_score` counts required-skill occurrences across project titles and their technologies, then adds a 15-point domain bonus, capped at 95
 - `exp_score` is 85 for intern/entry roles, else 70 - seniority-aware
 - aggregate: `hard x 0.50 + project x 0.30 + exp x 0.20`
 - Every score field is bounded by `ge=0 / le=100` in the schema, so an out-of-range LLM response fails validation and triggers the self-repair path rather than silently corrupting the UI ring.
 - Gap analysis is deductive, not generative: `missing_skills` is the literal set difference between the JD requirement list and the verified profile, which is precisely the Module V inference requirement.

Deterministic Fallback Path

- `_calculate_heuristic_match()` runs with no LLM at all:
 - `required_skills` defaults to ['Python'] when the feed supplied none
 - emits 3 concrete recommendations: lead with matched skills, address gaps via coursework or GitHub evidence, and rephrase bullets toward metrics

Presentation Talking Points

- **1.** Show the ATS ring per job card and the matched (green) vs missing (red) pill rows.
 - **2.** Write the 50/30/20 formula on the board - it is the single most exam-relevant idea in the project and directly implements heuristic evaluation.
 - **3.** Explain bidirectional substring matching with a live example, since it is the detail that makes the set intersection actually work on messy job titles.
 - **4.** Stress the honesty rule: missing skills are reported, never silently closed. This is what stops Agent 5 from inventing credentials.
-

Agent 5 — TailorAgent — What This Agent Does

Owens the synthesis stage and the final document engine. This member rewrites the candidate's summary, re-prioritises skills toward the JD, and sharpens project and experience bullets under a strict anti-hallucination policy, then compiles the validated result into an ATS-parseable PDF.

Owned Files

File	Lines	Responsibility
agents/tailor_agent.py	178	The agent, its feedback-repair loop, and its deterministic fallback.
tools/pdf_generator.py	213	ReportLab ATS-safe PDF compiler with a 6-style typography system.
schemas/tailoring_schema.py	101	TailoredResume plus TailoredBullet / TailoredExperience / TailoredProject (VerificationIssue and VerificationReport are co-owned with Member 6).

Tools, Libraries & Schemas Used

Dependency	How It Is Used
ReportLab	SimpleDocTemplate + Platypus flowables for PDF compilation
schemas/resume_schema	CandidateProfile and TechnicalSkills as the source
schemas/job_schema	JobListing required skills for keyword alignment
schemas/analysis_schema	MatchReport matched/missing skills as the target
schemas/tailoring_schema	TailoredResume is the output contract
agents/base_agent	call_structured_llm() inherited transport

Detailed Workflow

STAGE 4 TRIGGER: 'Synthesize & Audit Tailored Resumes' (ui/app.py:587)

```
[1] TailorAgent.tailor_resume(profile, job, match_report,
                             auditor_feedback=None)          (agent L29)
```

```
[2] SYSTEM PROMPT declares the ANTI-HALLUCINATION POLICY:
    1. GROUNDING RULE - may rephrase, reorder and emphasise real
       projects, skills and experience; MUST NOT invent companies,
       unearned degrees, fake years, or ungrounded technologies
    2. KEYWORD ALIGNMENT - integrate matched JD keywords ONLY where
       the candidate actually used related tools
    3. SUMMARY ALIGNMENT - 2-3 line summary targeted at this company
    4. SKILL PRIORITIZATION - move JD-critical tech to the front of
       each skill list
    5. if auditor_feedback is present, MUST correct every flagged
       violation immediately
```

```
[3] USER PROMPT assembles the full working set:
    candidate name, original summary, original skills,
    experience and projects as model_dump() dicts,
    target job + company, required JD skills,
    MATCHED skills and MISSING skills from the MatchReport
```

```
[4] *** REFLECTION HOOK ***          (agent L65-66)
    if auditor_feedback:
        user_prompt += 'CRITICAL AUDITOR FEEDBACK TO FIX:\n' + feedback
    This single append is what turns the agent from a one-shot generator
    into a participant in Member 6's producer-critic loop.
```

```
[5] call_structured_llm(schema=TailoredResume) with failover
```

```
[6] POST-PROCESSING (agent L76-82)
    contact / education / certifications are COPIED VERBATIM from the
    original profile - the tailor is never allowed to restate these
    ats_score_before = match_report.overall_match_score
    ats_score_after  = min(before + 18, 96)  # projected keyword gain

[7] OUTPUT: TailoredResume, each bullet retaining BOTH
    original_text and tailored_text (a per-bullet audit trail)

[8] compile_tailored_pdf(tailored_resume) -> bytes (pdf_generator.py:13)
    SimpleDocTemplate, Letter, 0.5in margins, BytesIO buffer
    ATS-SAFE CHOICES that make the PDF machine-readable:
    - single column, never multi-column or table-based content
    - base-14 Helvetica / Helvetica-Bold only (no embedded fonts)
    - no images, no text boxes, no tables for resume data
    SIX SECTION STORIES emitted in order:
    1. Header: uppercase name + pipe-joined contact + HRFlowable rule
    2. PROFESSIONAL SUMMARY
    3. TECHNICAL SKILLS matrix (5 labelled category lines)
    4. WORK EXPERIENCE - role / company / date+location header
    5. KEY PROJECTS - title + [Tech: ...] annotation
    6. EDUCATION, then CERTIFICATIONS
    6 custom ParagraphStyles on a restrained hex palette
    (#1A202C, #2B6CB0, #2D3748), keepWithNext on headings so a
    section title can never be orphaned from its content,
    hanging-indent bullets (leftIndent 14 / firstLineIndent -10)
```

Core Logic & Algorithms to Explain

- The +18 projected ATS gain is a deliberate, capped model: it reflects the real keyword-density improvement from reordering and rephrasing, but the cap at 96 prevents an inflated marketing number.
- Copying contact, education and certifications verbatim is a structural guarantee rather than a prompt request - the tailor has no opportunity to hallucinate them.
- Storing original_text alongside tailored_text in every TailoredBullet gives Agent 6 an exact diff surface to audit, which is why the CSP check is tractable.
- The PDF's ATS safety is an engineering decision, not cosmetics: many ATS parsers fail on custom embedded fonts and multi-column layouts, so base-14 single-column output is what makes the score improvements real.

Deterministic Fallback Path

- `_build_deterministic_tailored_resume()` (agent L86-178) activates offline:
 - sorts languages and frameworks so JD-matching skills sort to index 0
 - via `key=lambda s: 0 if s.lower() in jd_skills_lower else 1`
 - wraps every bullet in a TailoredBullet carrying matched skill keywords
 - synthesises a targeted 2-sentence summary naming the exact job and company
- PDF compilation never needs an LLM, so Stage 5 always produces a download.

Presentation Talking Points

- **1.** Open the downloaded PDF and explain why it looks deliberately plain: that restraint IS the ATS optimisation.
 - **2.** Contrast base-14 fonts and single-column layout against a typical designed resume that most ATS parsers silently drop.
 - **3.** Highlight the reflection hook at agent L65-66 - two lines of code are the seam where the whole producer-critic architecture attaches.
 - **4.** Show the before/after ATS delta badge and explain the +18 projection with its cap.
 - **5.** Note that contact and education are copied verbatim: a structural safeguard rather than a prompt instruction.
-

Agent 6 — VerifierAgent + Orchestrator + LLM Platform — What This Agent Does

Owens the safety and coordination layer. This member implements the adversarial auditor that enforces the subset constraint between the tailored resume and the original profile, runs the producer-critic reflection loop to convergence, and maintains the shared multi-provider LLM platform every other agent depends on.

Owned Files

File	Lines	Responsibility
agents/verifier_agent.py	133	The adversarial auditor plus the deterministic CSP checker.
agents/tailoring_orchestrator.py	83	The Tailor/Verifier reflection loop and convergence control.
agents/base_agent.py	482	Shared LLM platform: 3-provider failover, circuit breaker, self-repair.
config/settings.py	98	Env loading, key validation, model registries, retry and loop tuning.

Tools, Libraries & Schemas Used

Dependency	How It Is Used
schemas/resume_schema	CandidateProfile as immutable ground truth
schemas/tailoring_schema	VerificationReport + VerificationIssue output
agents/tailor_agent	The producer half of the loop
requests	OpenRouter + Groq OpenAI-compatible transport
google-genai	Tertiary Gemini SDK client
python-dotenv	load_dotenv() at settings import time
ui/app.py	status_callback streams the loop into the trace box

Detailed Workflow

```
PART A - VERIFIER (adversarial critic) (verifier L26)

[A1] audit(original_profile, tailored_resume)
[A2] SYSTEM PROMPT: 'solemn duty is to protect the candidate from
    resume fraud and recruiter blacklisting'. Four CSP constraints:
    1. NO INVENTED SKILLS - flag any skill absent from the original
    2. NO INVENTED EMPLOYERS OR ROLES - titles and companies must match
    3. NO INVENTED METRICS - no fabricated percentages or amounts
    4. OUTCOME - REVISION_NEEDED with exact feedback if any violation,
        else APPROVED
[A3] USER PROMPT is an explicit GROUND TRUTH vs CLAIMED SIDE-BY-SIDE:
    ORIGINAL: skills, experience, projects, certifications
    TAILORED: summary, prioritized skills, experiences, projects
[A4] call_structured_llm(schema=VerificationReport)

DETERMINISTIC CSP CHECK - _audit_grounding_heuristically() (L68-133)
    CONSTRAINT 1 skill subset inclusion (Tailored_Skills subset_of Original)
        for each tailored skill, bidirectional substring match against
        the original skill set; no match -> UNGROUNDED_SKILL issue
    CONSTRAINT 2 fabricated employer detection
        every tailored company must exist in the original experience set,
        else FALSE_EXPERIENCE issue
    if issues: status REVISION_NEEDED, is_grounding=False, risk MEDIUM,
        feedback_for_tailor concatenates every violation
    else: status APPROVED, is_grounding=True, risk LOW, no issues

PART B - ORCHESTRATOR (producer-critic loop) (tailoring_orchestrator L27)
```

```

[B1] run_tailoring_pipeline(profile, job, match_report,
                           max_retries=2, status_callback=None)
[B2] iteration = 0 ; current_feedback = None
[B3] while iteration <= max_retries:
    iteration += 1
    log('Tailor Agent drafting...')
    tailored = tailor_agent.tailor_resume(..., auditor_feedback=
                                         current_feedback)

    log('Verifier Agent auditing...')
    audit = verifier_agent.audit(profile, tailored)
    if audit.status == 'APPROVED':
        log('Audit PASSED'); break # CONVERGENCE
    current_feedback = audit.feedback_for_tailor # REFLECT
[B4] every log() fires BOTH print() and the injected status_callback,
    which the UI renders live in the execution trace box
[B5] returns (TailoredResume, VerificationReport, iteration_count)

```

PART C - LLM PLATFORM (inherited by ALL six agents) (base_agent.py)

```

[C1] PROVIDER CHAIN, strictly ordered by priority:
    1. OpenRouter - 9 free models, OpenAI-compatible, Bearer sk-or-*
    2. Groq - gpt-oss-120b / 20b, OpenAI-compatible, Bearer gsk_*
    3. Gemini - google-genai SDK, response_mime_type JSON
[C2] CLASS-LEVEL CIRCUIT BREAKER (L81-93)
    _tripped_providers is a class-level set shared by every agent
    instance for the whole process lifetime
[C3] PERMANENT vs TRANSIENT QUOTA CLASSIFICATION (L49-61)
    body markers 'free-models-per-day', 'add 10 credits',
    'monthly limit', 'daily limit' -> PERMANENT: trip the breaker
    immediately rather than burn retries that can never succeed
[C4] HTTP STATUS TRIAGE in _chat_completion_with_retry()
    429 -> exponential backoff 4.0s x 2^(n-1), then next model
    402/403 -> billing or auth fault -> trip breaker
    5xx -> backoff and retry, then next model
    other -> skip straight to next model
[C5] MODEL ROTATION: each provider holds an ordered roster; the
    transport walks it until one returns schema-valid content
[C6] SELF-REPAIR (L302-340): on Pydantic validation failure, resend the
    model's own bad JSON + the truncated validation error + the JSON
    Schema, at temperature 0.0, asking for a corrected object
[C7] JSON SANITISATION (L461-482): strip ```json fences, else slice the
    outermost brace pair by index
[C8] SCHEMA INJECTION: model_json_schema() is embedded in every prompt
    for OpenAI-compatible providers; Gemini uses JSON mime mode

```

Core Logic & Algorithms to Explain

- The grounding constraint is a genuine subset test, not a similarity score: the tailored skill set must be contained in the original skill set, which is exactly the Module IV CSP framing of the problem.
- Bidirectional substring matching is used deliberately, so 'React' is judged grounded by 'React.js' while a genuinely absent technology such as 'Kubernetes' is still caught.
- The circuit breaker is a class attribute rather than an instance attribute precisely so that exhausting one provider during Agent 1 also protects Agents 2 through 6 - a process-wide, not per-call, decision.
- The reflection loop is bounded (max 2 retries) and always returns the last produced artifact even on non-convergence, so the pipeline can never hang or return nothing.
- VerificationIssue carries suggested_correction, so the critic's feedback is actionable instruction rather than a bare rejection - that is what lets Agent 5 repair in one pass instead of oscillating.

Deterministic Fallback Path

- _audit_grounding_heuristically() is a complete offline CSP checker: no LLM, no network, deterministic verdict. This means the safety guarantee is never weakened by provider outage - arguably the most important resilience property in the system.
- The orchestrator itself has no fallback requirement; it is pure control flow.

Presentation Talking Points

- 1. Open with the constraint: write 'Tailored Resume subset_of Candidate Truth' on the board. Everything in this role defends that one line.
- 2. Show the live execution trace box during a Stage 4 run - it is the producer-critic loop made visible, which is the Module IX deliverable.

- **3.** Explain the difference between permanent and transient quota exhaustion, then show the class-level breaker log. This is the most technically interesting part of the platform and rarely understood.
- **4.** Emphasise that the deterministic CSP checker means hallucination protection still works with zero API keys - safety must not depend on availability.
- **5.** Close by tying the loop back together: five producers, one critic, one shared failover transport.

Shared Infrastructure & Cross-Cutting Concerns

These items are not owned by any single member but must be understood by all of them, because every agent boundary passes through them.

agents/base_agent.py — the Shared LLM Platform

- Provider order is fixed: OpenRouter (free) then Groq then Gemini. If one is unconfigured it is skipped; if one fails the next is tried automatically.
- **_tripped_providers** is a class-level set, so exhausting a provider in Member 1's stage protects every later stage in the same process.
- Permanent quota exhaustion (daily or monthly free-tier caps) is detected by matching response body markers and trips the breaker immediately, because retrying a permanently exhausted quota can never succeed.
- Transient failures (429, 5xx) use exponential backoff of $4.0s \times 2^{(n-1)}$ across up to 3 attempts per model, then advance to the next model in the roster.
- Every prompt embeds the JSON Schema of its target Pydantic model, so field descriptions in the schema classes double as prompt documentation.
- A schema validation failure triggers exactly one corrective repair call at temperature 0.0, where the model is shown its own invalid output and the error.

config/settings.py — Global Tuning

Constant	Value	Why It Matters
TEMPERATURE	0.2	Low temperature keeps factual resume content deterministic.
MAX_TOKENS	8192	Large multi-section JSON responses fit.
MAX_RETRIES_PER_MODEL	3	Bounded before advancing to the next model.
RETRY_BACKOFF_SECONDS	4.0	Base of the exponential backoff curve.
MAX_VERIFICATION_RETRIES	2	Bounds the Member 6 reflection loop.
MIN_ACCEPTABLE_ATS_SCORE	65	Threshold for an acceptable match.

ui/app.py — Where All Six Agents Are Invoked

UI Stage	Invokes	Session-State Gate
1. Resume Ingestion	Member 1	file_uploader != None
2. Career Strategy	Member 2	session_state.profile is set
3. Job Discovery	Members 3 and 4	session_state.selected_role is set
4. Tailoring and Audit	Members 5 and 6	session_state.discovered_jobs
5. PDF Delivery	Document engine	session_state.tailored_resumes

- Stage gating means the pipeline ordering is enforced by the UI itself, so a member can demo their stage in isolation once the prerequisite key is set.
- match_reports, tailored_resumes and generated_pdfs are keyed by job id, which is what allows several jobs to be tailored in a single batch pass.
- Uploading a new resume clears every downstream session-state key.
- The orchestrator's status_callback writes live into the trace box, so Member 6's reflection loop is visible to the audience while it runs.

Suggested Presentation Order

- Follow the data flow (Members 1 through 6) so the audience sees one continuous pipeline rather than six disconnected components.
- Have Member 3 present longest, since the search subsystem is the largest module.
- Have Member 6 present last, because the verification loop and the shared LLM platform tie all six agents back together.
- Run the live demo with the trace box visible so the producer-critic iterations are observable rather than explained.